

LABORATORY RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 5

Student Name: N. Samhavya

Roll No.: A24126552099

Date: 29-08-2026

1. TITLE

Develop a Dynamic Webpage Using DOM & JavaScript

2. AIM

To develop a dynamic **Expense Tracker** webpage using **HTML, CSS, JavaScript, and DOM manipulation** that allows users to add, display, delete, and calculate expenses dynamically.

3. OBJECTIVE

The objectives of this experiment are:

- To understand the concept of the **Document Object Model (DOM)**.
 - To access HTML elements using JavaScript.
 - To handle user interactions using event listeners.
 - To dynamically create and modify HTML elements.
 - To add and delete expense records dynamically.
 - To calculate and display the total expenses.
 - To perform input validation using JavaScript.
 - To use JavaScript arrays and objects for storing data.
 - To implement dynamic dark mode using CSS classes and JavaScript.
 - To understand how JavaScript can change webpage content without reloading the page.
-

4. THEORY

The **Document Object Model (DOM)** is a programming interface that represents an HTML document as a tree of objects. JavaScript can use the DOM to access, modify, add, or remove elements and content dynamically.

In this experiment, an **Expense Tracker** webpage is developed using HTML, CSS, and JavaScript. The HTML file provides the basic structure, CSS provides the visual appearance, and JavaScript handles the dynamic behavior.

An array named expenses is used to store expense objects. Each expense contains a **name** and an **amount**.

JavaScript accesses HTML elements using:

`document.getElementById()`

The application uses DOM methods such as:

- `getElementById()` – selects an HTML element using its ID.
- `createElement()` – creates a new HTML element dynamically.
- `appendChild()` – adds a new element to an existing element.
- `innerHTML` – modifies the HTML content of an element.
- `textContent` – changes the text content of an element.
- `classList.toggle()` – adds or removes a CSS class dynamically.

Event listeners are used to respond to user actions. When the **Add Expense** button is clicked, the entered expense is validated and added to the array. The expense list and total are then updated dynamically.

The application also provides a **Delete** button for each expense and a **Dark Mode** button that changes the webpage theme. The basic working flow is:

User Input → JavaScript Event → DOM Manipulation → Update Expense List → Calculate Total → Display Updated Result

5. ALGORITHM / PROCEDURE

1. Create a project folder for the Expense Tracker application.
2. Create an HTML file named `index.html`.
3. Create a CSS file named `style.css`.
4. Create a JavaScript file named `script.js`.
5. Design the webpage using HTML elements.
6. Add input fields for expense name and expense amount.
7. Add an **Add Expense** button.
8. Create a section to display the total expenses.
9. Create an unordered list to display expense records.
10. Create a theme button for switching between light and dark modes.
11. Initialize an empty JavaScript array named `expenses`.
12. Access the required HTML elements using `document.getElementById()`.
13. Create the `addExpense()` function to process user input.
14. Validate the expense name and amount.
15. Create an expense object containing the name and amount.
16. Add the expense object to the `expenses` array using `push()`.
17. Display the updated expenses using DOM manipulation.
18. Calculate the total amount using the `calculateTotal()` function.

19. Implement deleteExpense() to remove an expense using splice().
20. Add event listeners to the Add Expense and Theme buttons.
21. Use classList.toggle() to switch between light and dark themes.
22. Execute the webpage in a browser and verify the dynamic operations.

6. PROGRAM

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Expense Tracker</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="container">
    <header>
      <h1>Expense Tracker</h1>
      <button id="themeButton"> 🌙 </button>
    </header>
    <section class="summary">
      <h2>Total Expenses</h2>
      <p id="total">₹0</p>
    </section>
    <section class="form-section">
      <h2>Add Expense</h2>
      <input type="text" id="expenseName" placeholder="Expense name">
      <input type="number" id="expenseAmount" placeholder="Amount">
      <button id="addButton">Add Expense</button>
    </section>
    <section class="expenses-section">
      <h2>Expenses</h2>
      <ul id="expenseList"></ul>
    </section>
  </div>
  <script src="script.js"></script>
</body>
</html>
```

style.css

```

* { margin:
0; padding:
0;
    box-sizing: border-box;
}
body {
    font-family: Arial, sans-serif;
background: #f4f4f4;
    color: #222;
}
.container { width:
90%; max-width:
600px;
    margin: 40px auto;
}
header {
    display: flex; justify-content:
space-between; align-items:
center;
    margin-bottom: 25px;
}
header h1 {
    font-size: 30px;
}
#themeButton
{ border: none;
background: #222;
color: white;
padding: 10px 14px;
border-radius: 8px;
    cursor: pointer;
}
.summary,
.form-section, .expenses-section
{ background: rgb(246, 248,
241); padding: 20px; margin-
bottom: 20px; border-radius:
12px;
    box-shadow: 0 4px 12px rgba(0, 0, 0, 0.08);
}
.summary p {
    font-size: 28px;
font-weight: bold;
margin-top: 10px;
}

```

```

.form-section    input
{
    width: 100%;
padding:         12px;
margin-top:      10px;
border: 1px solid #ccc;
    border-radius: 8px;
}
#addButton
{   width: 100%;
margin-top: 15px;
padding: 12px;
border: none;
border-radius: 8px;
background:
#3299a2; color:
white; cursor:
pointer;
}
#addButton:hover {
    opacity: 0.8;
}
#expenseList
{   list-style: none;
margin-top: 15px;
}
#expenseList li {
    display: flex; justify-content:
space-between; align-items:
center; padding: 12px;
margin-bottom: 10px;
background: #f5f5f5;
    border-radius: 8px;
}
/* Dark mode */
body.dark
{   background:
#181818;
    color: white;
}
body.dark        .summary,
body.dark        .form-section,
body.dark .expenses-section
{
    background: #242424;
}

```

```

body.dark #expenseList li {
  background: #333;
}
body.dark input
{ background: #333;
color: white;
border-color: #555;
}
body.dark #themeButton
{ background: white;
color: #222;
}

```

script.js

```

// Store all expenses
let expenses = [];
// Get HTML elements const expenseName =
document.getElementById("expenseName"); const expenseAmount =
document.getElementById("expenseAmount"); const addButton =
document.getElementById("addButton"); const expenseList =
document.getElementById("expenseList"); const total =
document.getElementById("total");
const themeButton = document.getElementById("themeButton");
// Add an expense function addExpense()
{ const name = expenseName.value; const
amount = Number(expenseAmount.value);
// Check if inputs are empty if (name
=== "" || amount <= 0)
{ alert("Please enter a valid
expense."); return;
}
const expense =
{ name: name,
amount: amount
};
expenses.push(expense);
displayExpenses(); calculateTotal();
expenseName.value = "";
expenseAmount.value = "";
}
// Display expenses function displayExpenses()
{ expenseList.innerHTML = "";
expenses.forEach(function(expense, index)
{ const li = document.createElement("li");
li.innerHTML = `

```

```

        <span>
            ${expense.name} - ₹${expense.amount}
        </span>
        <button onclick="deleteExpense(${index})">
            Delete
        </button>
    `;
    expenseList.appendChild(li);
});
}
// Calculate total expenses
function calculateTotal()
{
    let totalAmount = 0;
    expenses.forEach(function(expense) {
        totalAmount = totalAmount + expense.amount;
    });
    total.textContent = "₹" + totalAmount;
}
// Delete an expense function
deleteExpense(index)
{
    expenses.splice(index, 1);
    displayExpenses();
    calculateTotal();
}
addButton.addEventListener("click", addExpense);
themeButton.addEventListener("click", function()
{
    document.body.classList.toggle("dark");
});

```

7. CODE EXPLANATION

Code / Element	Definition
let expenses = []	Creates an empty array to store expense records.
document.getElementById()	Selects HTML elements using their unique IDs.
.value	Gets the value entered in an input field.
Number()	Converts the entered amount into a number.
if	Validates the entered expense data.
alert()	Displays a validation message to the user.

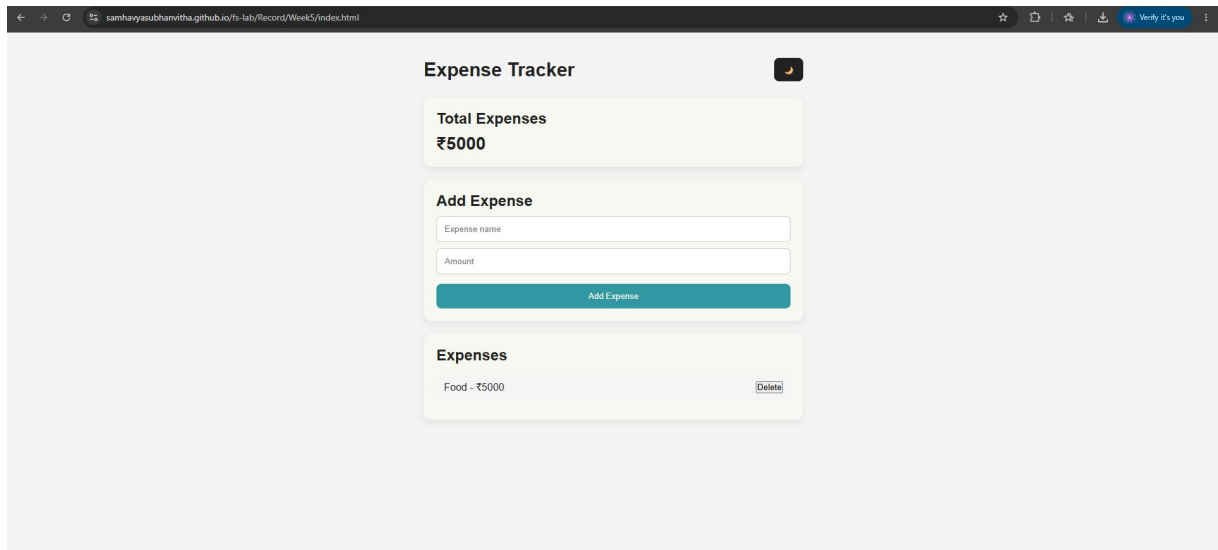
<code>const expense = {}</code>	Creates an object containing expense information.
<code>push()</code>	Adds a new expense object to the array.
<code>displayExpenses()</code>	Displays all stored expenses on the webpage.
<code>innerHTML</code>	Clears or modifies the HTML content of an element.
<code>forEach()</code>	Iterates through every expense in the array.
<code>document.createElement()</code>	Creates a new HTML element dynamically.
<code></code>	Represents an individual expense item in the list.
<code>appendChild()</code>	Adds a dynamically created element to the webpage.
<code>calculateTotal()</code>	Calculates the total amount of all expenses.
<code>textContent</code>	Updates the total expense displayed on the page.
<code>splice()</code>	Removes an expense from the array.
<code>addEventListener()</code>	Executes a function when a specified event occurs.
<code>classList.toggle()</code>	Adds or removes the dark class from the body.
<code>body.dark</code>	Applies CSS styles when dark mode is active.
<code>onclick</code>	Calls the delete function when the Delete button is clicked.

8. TEST CASES AND EXECUTION DATA

The Expense Tracker webpage was executed in a web browser and tested for adding, displaying, deleting, calculating expenses, input validation, and theme switching.

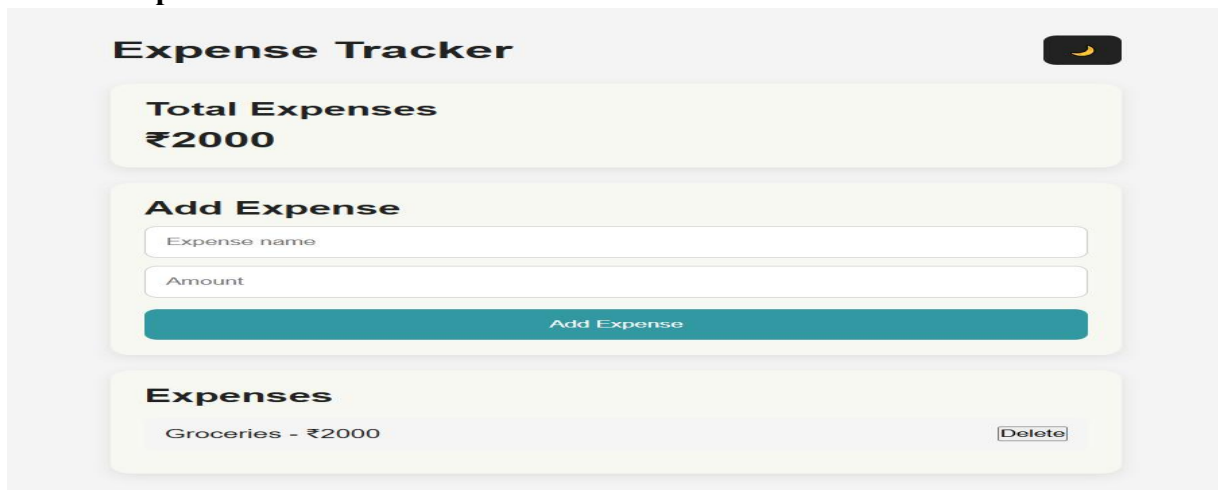
Test Case	TC-01
Test / Input	Open the Dynamic Expense Tracker webpage
Expected Result	Expense Tracker interface should be displayed with Total Expenses, Add Expense form, Expenses list, Delete button, and theme button.
Actual Result	Expense Tracker webpage was displayed correctly with all required components.
Status	Pass

TC-01 Output:



Test Case	TC-02
Test / Input	Add an expense with a valid expense name and amount
Expected Result	The new expense should be added to the Expenses list and the Total Expenses amount should be updated.
Actual Result	The expense was added successfully and the total expense was updated dynamically.
Status	Pass

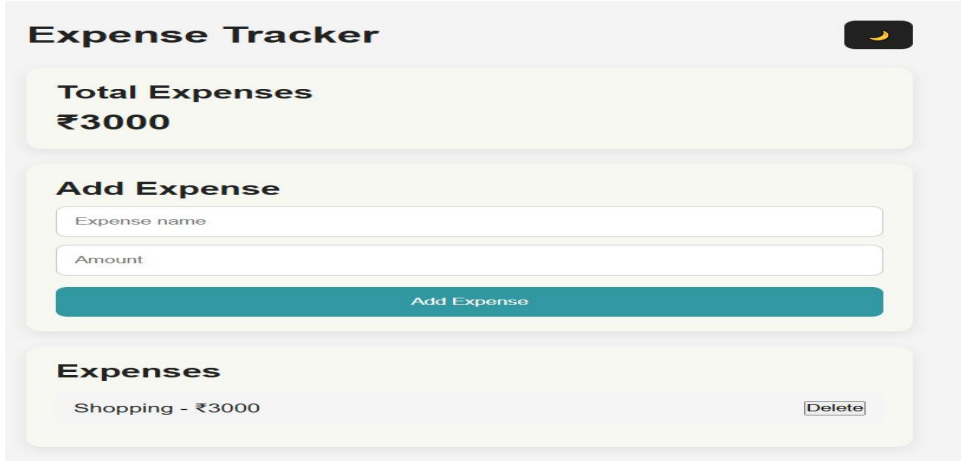
TC-02 Output:



Test Case	TC-03
Test / Input	Delete an existing expense by clicking the Delete button
Expected Result	The selected expense should be removed from the Expenses list and the total amount should be recalculated.

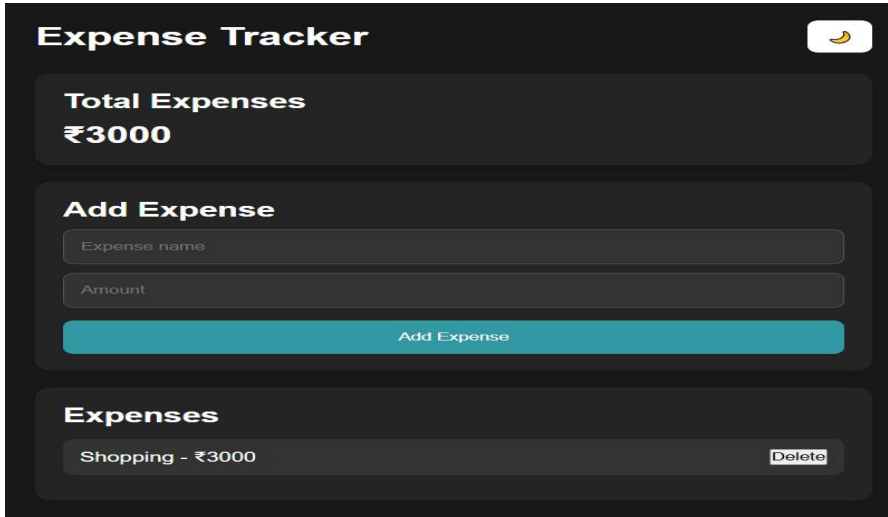
Actual Result	The expense was deleted successfully and the total was recalculated correctly.
Status	Pass

TC-03 Output:



Test Case	TC-04
Test / Input	Click the theme button
Expected Result	The webpage theme should switch between light and dark mode.
Actual Result	The theme was switched successfully.
Status	Pass

TC-04 Output:



9. OUTPUT

Output: Dynamic Expense Tracker Webpage

The following screenshot shows the complete execution of the **Expense Tracker dynamic webpage**. It displays the expense tracker interface with the total expense section, expense input fields, Add Expense button, expense list, Delete functionality, and theme button.

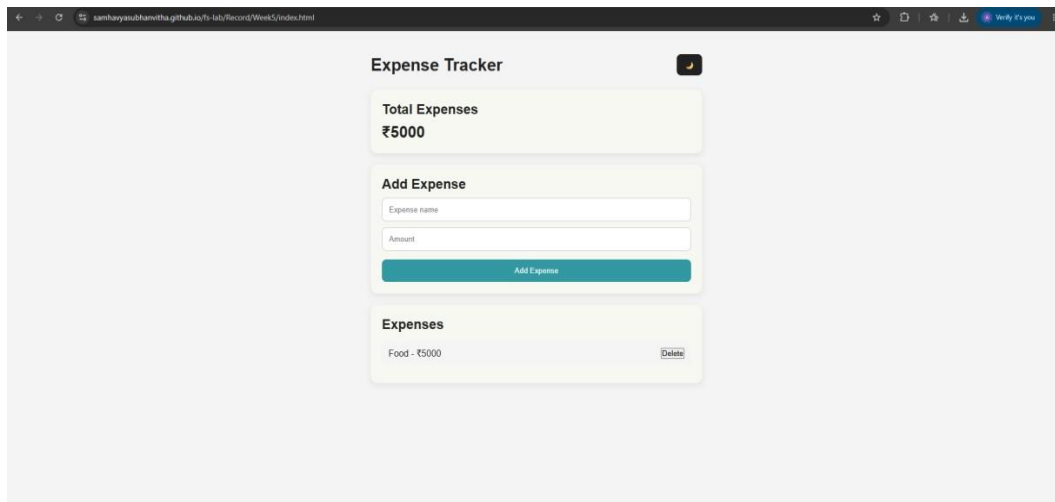


Figure 1: Output of Dynamic Expense Tracker Using DOM and JavaScript

10. RESULT

Thus, a dynamic **Expense Tracker webpage** was successfully developed using **HTML, CSS, JavaScript, and DOM manipulation**. The application successfully allows users to add, display, delete, and calculate expenses dynamically. Input validation, event handling, DOM element creation, array operations, and light/dark theme switching were implemented successfully. The webpage was executed and verified using multiple test cases, and the expected results were obtained.